



Métodos

Prof. Luís Guilherme de S. Lopes

Aula 9

- Conceito de Métodos
- Estrutura
- Como e onde usar em código

Métodos

- Permitem que você organize e reutilize o código de forma eficiente, evitando a repetição e tornando o programa mais legível e fácil de manter;
- Usados em todos os aspectos de um programa, desde operações simples até a implementação de algoritmos complexos;
 - Bloco de código que realiza uma tarefa específica.

```

opaqueV = "" if not os.path.exists(path):
    import shutil if os.path.exists("Input4RTAvTEST/"):
        shutil.rmtree("Input4RTAvTEST/")
    os.mkdir(path)
    for line in content:
        searchObj = re.search(r"([a-zA-Z0-9_]+)\.([a-zA-Z0-9_]+)", line)
        if searchObj:
            fileName = glob.glob("InputFolder/" + searchObj.group(1) + "." + searchObj.group(2) + "*")
            if searchObj:
                file_name = searchObj.group(1) + "." + searchObj.group(2) + ".txt"
                file_path = os.path.join(path, file_name)
                with open(file_path, "w") as f:
                    f.write(line)

```

Estrutura

- Um método em Java é composto por várias partes, cada uma com uma função específica

EXEMPLO:

```
retorno nomeDoMetodo(parâmetros) {  
    // Corpo do método  
    // Instruções que o método executa  
}
```

Estrutura

- Retorno (tipo de retorno): Especifica o tipo de dado que o método vai devolver depois que ele for executado. Pode ser qualquer tipo de dado, como `int`, `double`, `String`, etc. Se o método não devolver nada, usamos a palavra-chave `void`.
- nomeDoMetodo: É o nome do método. Deve ser descritivo e seguir as regras de nomenclatura de identificadores em Java (não pode começar com números, não pode ter espaços, etc.). A convenção é começar com letra minúscula e usar camelCase.

Estrutura

- Parâmetros: São valores que você pode passar para o método quando ele é chamado, para que ele possa trabalhar com esses valores. Parâmetros são opcionais; um método pode ter nenhum, um ou vários parâmetros.
- Corpo do método: É o bloco de código que define o que o método faz. As instruções dentro desse bloco são executadas quando o método é chamado.
- Modificador de Acesso: Define a visibilidade do método.

Exemplo

Método que soma dois números

```
public int somar(int a, int b) {  
    int resultado = a + b;  
    return resultado;  
}
```

Onde utilizar o Método?

Organização de Tarefas Repetitivas

Para tarefas que precisam ser realizada várias vezes em diferentes partes do código, você pode criar um método e chamá-lo sempre que necessário. Isso evita a duplicação de código e facilita a manutenção.

Onde utilizar o Método?

Exemplo: Calcular a média de notas de diferentes alunos.

```
import java.util.Scanner;

public class MediaNotas {

    // Método para calcular a média
    public static double calcularMedia(double[] notas) {
        double soma = 0;
        for (double nota : notas) {
            soma += nota;
        }
        return soma / notas.length;
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Digite o número de alunos: ");
        int numAlunos = scanner.nextInt();

        for (int i = 1; i <= numAlunos; i++) {
            System.out.print("Digite o número de notas para o aluno " + i + ": ");
            int numNotas = scanner.nextInt();
            double[] notas = new double[numNotas];

            for (int j = 0; j < numNotas; j++) {
                System.out.print("Digite a nota " + (j + 1) + ": ");
                notas[j] = scanner.nextDouble();
            }

            double media = calcularMedia(notas);
            System.out.println("A média do aluno " + i + " é: " + media);
        }

        scanner.close();
    }
}
```

Onde utilizar o Método?

Melhorar a Legibilidade do Código

Métodos ajudam a quebrar um código grande em pedaços menores e mais compreensíveis. Isso torna o código mais fácil de entender e de manter.

Exemplo: Em vez de colocar toda a lógica de um jogo em um único método `main`, você pode dividir as partes principais em métodos separados, como `iniciarJogo`, `processarEntrada`, e `verificarVencedor`.

Onde utilizar o Método?

Exemplo: Em vez de colocar toda a lógica de um jogo em um único método **main**, você pode dividir as partes principais em métodos separados, como **iniciarJogo**, **processarEntrada**, e **verificarVencedor**.

```
import java.util.Scanner;
import java.util.Random;

public class JogoAdivinhacao {

    // Método para iniciar o jogo
    public static void iniciarJogo() {
        System.out.println("Bem-vindo ao jogo de adivinhação!");
        System.out.println("Tente adivinhar o número que estou pensando, entre 1 e 100.");
    }

    // Método para processar a entrada do usuário
    public static int processarEntrada() {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Digite seu palpite: ");
        return scanner.nextInt();
    }

    // Método para verificar se o jogador venceu
    public static boolean verificarVencedor(int palpite, int numeroSecreto) {
        if (palpite == numeroSecreto) {
            System.out.println("Parabéns! Você acertou o número!");
            return true;
        } else if (palpite < numeroSecreto) {
            System.out.println("O número é maior que " + palpite);
        } else {
            System.out.println("O número é menor que " + palpite);
        }
        return false;
    }

    public static void main(String[] args) {
        iniciarJogo();

        Random random = new Random();
        int numeroSecreto = random.nextInt(100) + 1;
        boolean venceu = false;

        while (!venceu) {
            int palpite = processarEntrada();
            venceu = verificarVencedor(palpite, numeroSecreto);
        }

        System.out.println("Obrigado por jogar!");
    }
}
```

Onde utilizar o Método?

Reutilização de Código

Ao definir um método, você pode reutilizá-lo em várias partes do seu programa ou até em diferentes programas, sem precisar reescrever o código.

Onde utilizar o Método?

Exemplo: Converter uma temperatura de Celsius para Fahrenheit.

```
import java.util.Scanner;

public class ConversorTemperatura {

    // Método para converter Celsius para Fahrenheit
    public static double celsiusParaFahrenheit(double celsius) {
        return (celsius * 9/5) + 32;
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Solicita ao usuário a temperatura em Celsius
        System.out.print("Digite a temperatura em Celsius: ");
        double celsius = scanner.nextDouble();

        // Converte a temperatura para Fahrenheit
        double fahrenheit = celsiusParaFahrenheit(celsius);

        // Exibe o resultado
        System.out.println(celsius + "°C equivale a " + fahrenheit + "°F");

        scanner.close();
    }
}
```

Onde utilizar o Método?

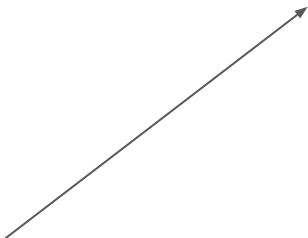
Encapsulamento e Modificação

Os métodos permitem encapsular a lógica de uma classe, mantendo os detalhes de implementação ocultos e fornecendo uma interface clara para interagir com essa lógica. Isso também facilita a modificação do comportamento de uma classe sem afetar o código externo que usa essa classe.

Onde utilizar o Método?

Exemplo: Em uma classe **ContaBancaria**, você pode ter métodos como **depositar**, **sacar**, e **obterSaldo**.

```
public class ContaBancaria {  
    private double saldo;  
  
    // Construtor  
    public ContaBancaria(double saldoInicial) {  
        this.saldo = saldoInicial;  
    }  
  
    // Método para depositar dinheiro na conta  
    public void depositar(double valor) {  
        if (valor > 0) {  
            saldo += valor;  
            System.out.println("Depósito de R$ " + valor + " realizado com sucesso.");  
        } else {  
            System.out.println("Valor de depósito inválido.");  
        }  
    }  
  
    // Método para sacar dinheiro da conta  
    public void sacar(double valor) {  
        if (valor > 0 && valor <= saldo) {  
            saldo -= valor;  
            System.out.println("Saque de R$ " + valor + " realizado com sucesso.");  
        } else {  
            System.out.println("Valor de saque inválido ou saldo insuficiente.");  
        }  
    }  
  
    // Método para obter o saldo atual da conta  
    public double obterSaldo() {  
        return saldo;  
    }  
  
    public static void main(String[] args) {  
        // Criação de uma conta bancária com saldo inicial de R$ 1000.00  
        ContaBancaria minhaConta = new ContaBancaria(1000.00);  
  
        // Exibe o saldo inicial  
        System.out.println("Saldo inicial: R$ " + minhaConta.obterSaldo());  
  
        // Deposita R$ 500.00 na conta  
        minhaConta.depositar(500.00);  
  
        // Exibe o saldo após o depósito  
        System.out.println("Saldo após depósito: R$ " + minhaConta.obterSaldo());  
    }  
}
```



```
// Tenta sacar R$ 200.00  
minhaConta.sacar(200.00);  
  
// Exibe o saldo após o saque  
System.out.println("Saldo após saque: R$ " + minhaConta.obterSaldo());  
  
// Tenta sacar um valor maior do que o saldo  
minhaConta.sacar(2000.00);  
  
// Exibe o saldo final  
System.out.println("Saldo final: R$ " + minhaConta.obterSaldo());  
}
```

Onde utilizar o Método?

Modularização do Código

Em programas complexos, a modularização é essencial. Métodos permitem que você divida um problema grande em partes menores, cada uma resolvida por um método específico.

Onde utilizar o Método?

Exemplo: Em um sistema de gerenciamento de pedidos, você pode ter métodos para **adicionarItem**, **removerItem**, **calcularTotal**, etc.

```
import java.util.ArrayList;
import java.util.List;

class Item {
    private String nome;
    private double preco;

    public Item(String nome, double preco) {
        this.nome = nome;
        this.preco = preco;
    }

    public String getNome() {
        return nome;
    }

    public double getPreco() {
        return preco;
    }
}

class Pedido {
    private List<Item> itens;

    public Pedido() {
        this.itens = new ArrayList<>();
    }

    // Método para adicionar um item ao pedido
    public void adicionarItem(Item item) {
        itens.add(item);
        System.out.println("Item " + item.getNome() + " adicionado ao pedido.");
    }

    // Método para remover um item do pedido pelo nome
    public void removerItem(String nomeItem) {
        boolean itemRemovido = false;
        for (Item item : itens) {
```

```
            if (item.getNome().equalsIgnoreCase(nomeItem)) {
                itens.remove(item);
                System.out.println("Item " + nomeItem + " removido do pedido.");
                itemRemovido = true;
                break;
            }
        }
        if (itemRemovido) {
            System.out.println("Item " + nomeItem + " não encontrado no pedido.");
        }
    }

    // Método para calcular o total do pedido
    public double calcularTotal() {
        double total = 0;
        for (Item item : itens) {
            total += item.getPreco();
        }
        return total;
    }

    // Método para exibir os itens no pedido
    public void exibirItens() {
        if (itens.isEmpty()) {
            System.out.println("O pedido está vazio.");
        } else {
            System.out.println("Itens no pedido:");
            for (Item item : itens) {
                System.out.println("- " + item.getNome() + " : R$ " + item.getPreco());
            }
        }
    }
}
```

public class SistemaDePedidos {

```
    public static void main(String[] args) {
        // Criando alguns itens
        Item item1 = new Item("Pizza", 35.00);
        Item item2 = new Item("Refrigerante", 8.00);
        Item item3 = new Item("Sobremesa", 12.50);

        // Criando um pedido
        Pedido meuPedido = new Pedido();

        // Adicionando itens ao pedido
        meuPedido.adicionarItem(item1);
        meuPedido.adicionarItem(item2);
        meuPedido.adicionarItem(item3);

        // Exibindo os itens do pedido
        meuPedido.exibirItens();

        // Calculando e exibindo o total do pedido
        System.out.println("Total do pedido: R$ " + meuPedido.calcularTotal());

        // Removendo um item do pedido
        meuPedido.removerItem("Refrigerante");

        // Exibindo os itens do pedido após a remoção
        meuPedido.exibirItens();

        // Calculando e exibindo o total atualizado do pedido
        System.out.println("Total do pedido após remoção: R$ " +
            meuPedido.calcularTotal());
    }
}
```

Onde utilizar o Método?

Trabalhar com Estruturas de Dados

Você pode criar métodos para manipular estruturas de dados como listas, arrays, mapas, etc., tornando o código mais claro e reutilizável.

Onde utilizar o Método?

Exemplo: Ordenar uma lista de números.

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class OrdenacaoLista {

    public static void main(String[] args) {
        // Criando uma lista de números inteiros
        List<Integer> numeros = new ArrayList<>();
        numeros.add(42);
        numeros.add(17);
        numeros.add(8);
        numeros.add(99);
        numeros.add(23);

        // Exibindo a lista antes da ordenação
        System.out.println("Lista antes da ordenação: " + numeros);

        // Ordenando a lista em ordem crescente
        Collections.sort(numeros);

        // Exibindo a lista após a ordenação
        System.out.println("Lista após a ordenação (crescente): " + numeros);

        // Ordenando a lista em ordem decrescente
        Collections.sort(numeros, Collections.reverseOrder());

        // Exibindo a lista após a ordenação decrescente
        System.out.println("Lista após a ordenação (decrescente): " + numeros);
    }
}
```

Onde utilizar o Método?

Implementação de Algoritmos

Métodos são ideais para implementar algoritmos específicos que podem ser complexos, como algoritmos de ordenação, busca, ou cálculo matemático.

Onde utilizar o Método?

Exemplo: Implementar a busca binária em um array.

```
import java.util.Arrays;

public class BuscaBinaria {

    // Método para implementar a busca binária
    public static int buscaBinaria(int[] array, int chave) {
        int inicio = 0;
        int fim = array.length - 1;

        while (inicio <= fim) {
            int meio = (inicio + fim) / 2;

            if (array[meio] == chave) {
                return meio; // A chave foi encontrada
            } else if (array[meio] < chave) {
                inicio = meio + 1; // A chave está à direita
            } else {
                fim = meio - 1; // A chave está à esquerda
            }
        }

        return -1; // A chave não foi encontrada
    }

    public static void main(String[] args) {
        // Um array ordenado onde a busca binária será realizada
        int[] numeros = {3, 8, 15, 23, 42, 55, 78, 90};

        // Valor que queremos buscar
        int chave = 23;

        // Realizando a busca binária
        int resultado = buscaBinaria(numeros, chave);

        // Exibindo o resultado da busca
        if (resultado != -1) {
            System.out.println("O valor " + chave + " foi encontrado na posição " + resultado + " do array.");
        } else {
            System.out.println("O valor " + chave + " não foi encontrado no array.");
        }
    }
}
```

Onde utilizar o Método?

Interação com Usuário

Métodos também podem ser usados para organizar a interação com o usuário, como exibir menus, capturar entradas, e processar escolhas.

Resolução de Problemas Complexos

Quando você precisa resolver problemas complexos, pode dividir o problema em subproblemas, cada um resolvido por um método separado.

Onde utilizar o Método?

Exemplo: Resolver uma equação matemática complexa por meio de métodos que resolvem partes da equação.

```
public class EquacaoComplexa {  
  
    // Método para multiplicar dois números: a * b  
    public static double multiplicar(double a, double b) {  
        return a * b;  
    }  
  
    // Método para subtrair dois números e elevar ao quadrado: (c - d)^2  
    public static double subtrairEQuadrado(double c, double d) {  
        return Math.pow(c - d, 2);  
    }  
  
    // Método para somar dois números: (parte1 + parte2)  
    public static double somar(double parte1, double parte2) {  
        return parte1 + parte2;  
    }  
  
    // Método para dividir dois números: resultadoFinal / e  
    public static double dividir(double resultado, double e) {  
        if (e != 0) {  
            return resultado / e;  
        } else {  
            throw new ArithmeticException("Divisão por zero não é permitida.");  
        }  
    }  
  
    // Método principal para resolver a equação  
    public static double resolverEquacao(double a, double b, double c, double d, double e) {  
        double parte1 = multiplicar(a, b);  
        double parte2 = subtrairEQuadrado(c, d);  
        double resultadoSoma = somar(parte1, parte2);  
        return dividir(resultadoSoma, e);  
    }  
  
    public static void main(String[] args) {  
        // Valores de entrada para a equação  
        double a = 2.0;  
        double b = 3.0;  
        double c = 10.0;  
        double d = 4.0;  
        double e = 5.0;  
  
        // Resolvendo a equação  
        double resultado = resolverEquacao(a, b, c, d, e);  
  
        // Exibindo o resultado  
        System.out.println("O resultado da equação é: " + resultado);  
    }  
}
```

Prática

